

VAL Vendor Analysis Assistant

Windows local deployment guide

This guide runs the current Streamlit application locally on Windows. The application authenticates with Azure using `DefaultAzureCredential`, then invokes the deployed VAL agent through the Azure AI Foundry Responses API. It requires Azure CLI sign-in or another supported Azure identity.

1. Prerequisites

Install:

- Python 3.11 or newer from <https://www.python.org/downloads/windows/>
- Git for Windows from <https://git-scm.com/download/win> (only if you need to clone the repository)

During Python installation, select Add Python to PATH.

2. Get the application

Open PowerShell and either clone the repository or change to the project folder:

```
git clone <repository-url>
cd VALAgent_Foundry
```

If the application work is on a feature branch, check out that branch before installing dependencies:

```
git checkout cursor/vendor-analysis-chat-a1f3
```

3. Create and activate a virtual environment

```
py -3 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
```

If PowerShell blocks the activation script, allow it only for the current terminal session:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1
```

4. Configure Azure AI Foundry

Create a local configuration file:

```
Copy-Item .env.example .env
notepad .env
```

Replace every placeholder in `.env`:

```
AZURE_AIPROJECT_ENDPOINT=https://your-resource.services.ai.azure.com/api/projects/your-project
OPENAI_API_VERSION=2025-04-01-preview
AZURE_VAL_AGENT_NAME=your-val-agent-name
AZURE_VAL_AGENT_VERSION=1
```

Use the project endpoint and deployed agent name/version from Azure AI Foundry. The API version must support agent references through the Responses API.

Keep `.env` private. It is ignored by Git and must never be committed or shared.

5. Authenticate with Azure CLI

Install Azure CLI if needed, then authenticate with an Azure identity that can invoke the Foundry agent:

```
az login --use-device-code
```

Complete sign-in in the displayed browser flow. The identity needs access to the target Foundry project and agent.

6. Run the application

With the virtual environment still active:

```
streamlit run app.py
```

Open the URL Streamlit prints, normally:

```
http://localhost:8501
```

Stop the server with `Ctrl+C`.

7. Verify the application

1. The sidebar should show Foundry agent configuration detected.
2. Confirm the agent information lists Foundry-managed agent.
3. Click New Conversation.
4. Send a prompt such as `Summarize the highest contract risks.`
5. Verify the assistant responds and that a follow-up question retains the prior chat context.

Troubleshooting

Security checklist

- Store secrets only in `.env` or a managed secret store.
- Do not place credentials in `app.py`, `.env.example`, screenshots, tickets, or commits.
- Use a least-privilege Azure identity to access the Foundry project.
- Restrict Foundry project network access according to your organization's security requirements.